ENGINEERING REFERENCE & SPECIFICATION

ArchViz Autonomous: 24/7 Cloud Engineer Agent

Complete Architecture, Telemetry Pipeline, Handoff Contracts & 12-Week Execution Roadmap

PLATFORM INTEGRATION LEAD
Krish (AWS Ingestion & Pipeline)

AI LEAD 1: DIAGNOSTICS & RCA
Gargi (Root-Cause Engine)

AI LEAD 2: REMEDIATION & SAFETY
Sweta (Terraform & Simulator)

1. The Big Picture: What We Are Building ("In Short")

ArchViz started as a modern visual cloud design tool that lets engineers draw cloud architectures on an interactive canvas, test latency and costs, and deploy infrastructure via Terraform.

ArchViz Autonomous is the next-generation evolution: an **autonomous 24/7 AI Cloud Engineer** that continuously monitors customer AWS infrastructure. If a database maxes out on connections, a container enters a crash loop, a security group is left open to the world, or an API begins timing out, the system automatically detects the incident, finds the exact root cause, writes a verified Terraform fix, and alerts the user with a single **[Approve & Apply Fix]** button.

1. Krish's Engine (YOU)

Platform Integration & Telemetry: Connects to the user's AWS account, catches alarms, collects 30m CloudWatch logs + 60m CloudTrail events, attaches the visual Canvas Graph, and hands it over.

2. Gargi's Engine

Diagnostic AI (RCA): Deduplicates cascading alarms, uses RAG over 25 AWS troubleshooting runbooks, pinpoints the root failing resource, and computes confidence score (0-100%).

3. Sweta's Engine

Remediation & Safety: Takes Gargi's diagnosis, writes the Terraform code diff + AWS SDK command, passes 3-Gate safety checks, and runs pre-flight latency/cost simulations.

KRISH'S CLEAR BOUNDARY

Your responsibility is strictly **getting the real logs and events from the user's AWS account** and delivering a clean, standardized data packet to Gargi. You do not write Gargi's diagnostic prompts or Sweta's Terraform fix code. You build the robust highway that delivers live AWS truth into their AI modules.

2. What We Have Built & Delivered ("In Detail")

All your code has been cleanly packaged and verified in the new standalone repository: `C:\Users\kumar\OneDrive\Desktop\ArchViz_Autonomous`.

1. **Dynamic Multi-Tenant AWS Account Access (`src/aws/credentials.ts`):**
   Users can authenticate either via direct IAM keys (`AWS_ACCESS_KEY_ID` / `AWS_SECRET_ACCESS_KEY`) or via enterprise cross-account **STS AssumeRole** (`RoleArn` + `ExternalId`). The system creates temporary 15-minute read-only sessions with zero credentials permanently stored.

2. **24/7 Autonomous Alarm Webhook Listener (`src/api/webhooks.ts`):**
   An HTTP endpoint `POST /api/webhooks/aws-alarm` that receives alerts from AWS EventBridge, CloudWatch Alarm state changes, AWS SNS topic notifications, or direct API triggers whenever an outage begins.

3. **Intelligent Log Group Auto-Discovery (`src/collector/cloudWatchCollector.ts`):**
   Instead of guessing, the collector inspects the failing AWS resource ARN and probes the user's AWS account using `DescribeLogGroups` for Lambda (`/aws/lambda/*`), RDS (`/aws/rds/instance/*`), ECS (`/ecs/*`), and API Gateway (`API-Gateway-Execution-Logs_*`), ensuring it finds real logs on live AWS accounts.

4. **Trailing 30-Minute CloudWatch Log Pre-Filter (`src/collector/logFilter.ts`):**
   CloudWatch logs are full of noise (`[INFO] Application started` , `GET /health 200 OK` , heartbeats). Your pre-filter strips out all noise lines and extracts only `ERROR` , `FATAL` , `WARN` , and `TIMEOUT` lines to keep Gargi's LLM context clean and cost-effective.

5. **Trailing 60-Minute CloudTrail Configuration Lookups (`src/collector/cloudTrailCollector.ts`):**
   Queries AWS CloudTrail for the trailing 60 minutes to uncover recent deployments and administrative changes (e.g., `AuthorizeSecurityGroupIngress` , `ModifyDBParameterGroup` , `RegisterTaskDefinition` , `UpdateService`). It extracts user identities and full JSON request parameters.

6. **ArchViz Canvas Graph Topology Attachment (`src/types/canvasGraph.ts`):**
   Attaches the user's architecture diagram topology (all `nodes` with `id` , `type` , `awsResourceId` , and `edges` with `source` and `target`). The resource indexer maps failing AWS resource ARNs directly to Canvas Node IDs so Gargi can trace upstream/downstream connections.

3. The 5 Pre-Built Target Failure Scenarios (For Offline Verification)

To allow Gargi and Sweta to develop and test their code offline without needing live AWS outages, your generator (`src/generator/sampleOutageGenerator.ts`) produces full test payloads for all 5 target failure scenarios:

Scenario	Trigger Alarm & Metric	Filtered Logs (30m) & CloudTrail Evidence (60m)	ArchViz Topology
1. RDS Bottleneck	`CPUUtilization: 96.4%` `DatabaseConnections: 498/500`	`FATAL: remaining connection slots are reserved for non-replication superuser connections` CloudTrail: `ModifyDBParameterGroup` (max_connections)	APIGW ➔ Lambda ➔ Aurora Postgres
2. Container CrashLoop	`ContainerCrashCount: 7` Task stop count > 3 in 10m	Container `Exit Code 137 (OutOfMemoryError)` - V8 JS Heap Limit Exceeded CloudTrail: `RegisterTaskDefinition` (memory reduced to 512MB)	ALB ➔ ECS Service ➔ Redis Cache
3. Security Group Drift	AWS Config Non-Compliant GuardDuty Portscan	Suspicious external SSH & Postgres connection handshakes CloudTrail: `AuthorizeSecurityGroupIngress` opening port 22/5432 to `0.0.0.0/0`	Internet GW ➔ Security Group ➔ RDS
4. API 5xx Timeouts	`5XXError Rate: 48%` `TargetResponseTime: 3450ms`	Lambda log: `Task timed out after 3.00 seconds` ALB: `504 Gateway Timeout` CloudTrail: `PutScalingPolicy` hitting max capacity	CloudFront ➔ ALB ➔ Lambda ➔ DynamoDB
5. FinOps Idle Waste	`VolumeTotalReadWriteOps: 0` `BytesOutToDestination: 0`	AWS Cost Anomaly Detection alert: unattached volume for 168 hours ($125/mo waste) CloudTrail: `DetachVolume` leaving volume unattached	Legacy VPC ➔ NAT Gateway & Orphaned EBS

4. How Gargi Will Link Her Engine With Your Code

Gargi connects to your code using a single standardized contract called **Handoff 1**. You deliver the raw data; her system consumes it.

> **THE EXACT HANDOFF 1 DATA CONTRACT (SRC/TYPES/ALERTPAYLOAD.TS)**
> Whenever an incident triggers, your backend service produces this exact TypeScript object and passes it to Gargi:

```
export interface AlertPayload {
  incidentId: string;
  triggeredAt: string;
  activeAlarms: Array<{
    alarmId: string;
    alarmName: string;
    nodeId: string;          // Maps directly to ArchViz canvas node ID
    awsResourceId: string;   // Full AWS ARN or physical ID
```

```
    metricName: string;
    metricValue: number;
    threshold: number;
    timestamp: string;
  }>;
  cloudWatchLogs30m: Array<{
    timestamp: string;
    nodeId: string;
    awsResourceId: string;
    level: 'ERROR' | 'FATAL' | 'WARN' | 'TIMEOUT';
    message: string;
  }>;
  cloudTrailEvents60m: Array<{
    eventId: string;
    eventTime: string;
    eventName: string;
    awsResourceId: string;
    userIdentity: string;
    requestParameters: Record<string, unknown>;
  }>;
  canvasGraph: {
    nodes: Array<{ id: string; type: string; awsResourceId: string; data: Record<string, unknown> }>;
    edges: Array<{ id: string; source: string; target: string }>;
  };
}
```

How Gargi Invokes Your Code (Two Clean Methods):

Option A: Direct Function Hook

In `src/handoff/gargiConnector.ts`, your system exports:
`handoffToGargiEngine(alertPayload)`
This directly invokes Gargi's function:
`runAutomatedRcaPipeline(alertPayload)`.
Gargi registers her implementation by calling
`registerGargiEngine(myRcaHandler)`.

Option B: REST Webhook / API Pull

If Gargi runs as a separate Python or Node microservice:
1. Your service can HTTP POST the payload to
`GARGI_RCA_WEBHOOK_URL`.
2. Or Gargi's service can fetch any incident payload by ID via:
`GET /api/handoff/incidents/:id`.

5. What Gargi Does With Your Data (Her Diagnostic Scope)

Once your backend feeds the `AlertPayload` into Gargi's function, her code executes four sequential tasks:

A **Graph Alert Deduplication:** When an RDS database fails, the connected Lambda and API Gateway also start throwing errors, creating 10+ alarms. Gargi uses the `canvasGraph` you provide to traverse downstream edges and find the single deepest root resource (collapsing 10 alarms into 1 root incident).

B **Runbook Search via RAG:** For that failing resource, her RAG retriever takes the pre-filtered logs and CloudTrail changes you gave her and queries her vector database of 25 AWS troubleshooting runbooks to find the root cause.

C **Confidence Scoring (0% to 100%):** She calculates a mathematical confidence score: 40% if an exact error code matches, 25% if a CloudTrail change happened shortly before the alarm, 20% if metrics crossed 90%, and 15% based on runbook similarity.

D **Handoff 2 to Sweta:** Gargi packages her diagnosis into `DiagnosisReport` (containing `failingNodeId`, `scenarioType`, `confidenceScore`, `rootCauseSummary`, and `recommendedFixIntent`) and passes it to Sweta.

6. What We Still Have To Do (Roadmap & Next Steps)

Here is the roadmap from the 12-Week AI Engineering Schedule, clearly showing where your project is now and what comes next:

Sprint Window	What Krish (YOU) Owns	What Gargi & Sweta Own	Status
Weeks 1–2 (Nov 1–14)	**AWS Telemetry Ingestion Pipeline:** Build live CloudWatch & CloudTrail collector, log filter, canvas graph adapter, and 5 offline scenarios.	Gargi: Graph alert deduplication & 25 runbook markdown files. Sweta: 3-Gate safety inspector.	`KRISH DONE (100%)`
Weeks 3–4 (Nov 15–30)	**Sandbox Webhook Integration:** Connect AWS EventBridge webhook listener to automatically trigger Gargi ➔ Sweta pipeline in sandbox.	Gargi: Runbook RAG retriever in `RagRetrieverNode.tsx`. Sweta: Terraform HCL diff & AWS SDK command generator.	`READY TO WIRE`
Weeks 5–6 (Dec 1–14)	**Interactive Approval Engine:** Wire Sweta's Verified Fix Package into Slack Block Kit and ArchViz Web UI with `[Approve & Fix]` buttons.	Gargi: Real-time incident logs in chat drawer. Sweta: Telemetry-calibrated pre-flight simulation.	`UPCOMING`
Weeks 7–8 (Dec 15–31)	**Beta User Onboarding:** Onboard first 5–10 beta companies and connect their read-only AWS accounts to the telemetry pipeline.	Gargi: 25 simulated faults in test VPC. Sweta: GitOps PR backport engine.	`UPCOMING`
Weeks 9–12 (Jan 1–31)	**Autonomous Cloud Agent Public Launch:** Convert audit users into paying 24/7 autonomous accounts. Automated fix	Gargi: Tuning noisy alerts from launch logs. Sweta: Expanding remediation templates to top 15 AWS services.	`UPCOMING`

Sprint Window	What Krish (YOU) Owns	What Gargi & Sweta Own	Status
	execution in AWS on approval.		

7. Krish's Immediate Next Tasks

> **KRISH'S NEXT ENGINEERING STEPS**
>
> 1. **Give Gargi the Handoff 1 Contract & Datasets:** Share `src/types/alertPayload.ts` and the 5 pre-generated JSON files in `datasets/` (or link her to the GitHub repo).
> 2. **Wire Gargi's Entry Hook:** When Gargi pushes her `runAutomatedRcaPipeline`, wire it into `src/handoff/gargiConnector.ts` so live alerts flow straight into her module.
> 3. **Prepare the Approval Execution Engine (Weeks 5–6):** Once Sweta delivers Handoff 3 (Verified Fix Package), build the 1-click execution routine that calls the AWS SDK v3 to apply the approved change to the user's AWS account.

8. Datasets for Gargi: What to Give Her & How to Get More

Gargi is asking for **datasets** because her AI pipeline (RAG vector search, alert deduplication, and confidence scoring) cannot be trained or tested without real, structured cloud telemetry.

A. Exactly What Files to Give Gargi (Already Prepared in `datasets/`):

Dataset File (in `datasets/`)	Target Failure Scenario	What It Contains for Gargi's AI
`alertPayload-rds_bottleneck.json`	RDS Database Bottleneck	4 alarms (RDS CPU 96%, Conn 498/500, Lambda Cascading, APIGW 504), 6 filtered logs (`FATAL: remaining connection slots reserved`), CloudTrail `ModifyDBParameterGroup`, and Canvas Graph (APIGW ➔ Lambda ➔ RDS).
`alertPayload-container_crashloop.json`	ECS/EKS Container CrashLoop	2 alarms (StopCount: 7, Unhealthy Hosts: 5), 5 logs (`Exit Code 137 OutOfMemoryError`, V8 JS heap out of memory), CloudTrail `RegisterTaskDefinition` (memory cut to 512MB), Canvas Graph (ALB ➔ ECS ➔ Redis).
`alertPayload-security_group_drift.json`	Security Group Drift	2 alarms (Config non-compliant, GuardDuty portscan), 4 logs (unauthorized SSH/DB handshakes), CloudTrail `AuthorizeSecurityGroupIngress` (0.0.0.0/0 on port 22/5432), Canvas Graph (Internet ➔ SG ➔ RDS).
`alertPayload-api_5xx_timeouts.json`	API 5xx Timeouts	3 alarms (TargetResponseTime: 3450ms, 5xx rate: 48%), 5 logs (Lambda `Task timed out after 3.00 seconds`, ALB 504), CloudTrail `PutScalingPolicy`, Canvas Graph (CloudFront ➔ ALB ➔ Lambda ➔ DynamoDB).
`alertPayload-finops_idle_waste.json`	FinOps Idle Waste	2 alarms (0 IOPS for 7 days, 0 bytes on NAT GW), 3 logs (AWS Cost Anomaly Detection alert: $125/mo waste), CloudTrail `DetachVolume`, Canvas Graph (Orphaned EBS & NAT Gateway).

B. How Gargi Uses These Datasets:

1. Graph Alert Deduplication

In `alertPayload-rds_bottleneck.json`, there are 4 separate alarms. Gargi feeds `canvasGraph` into her `findRootFailingNode(activeAlarms, graph)` function to prove her graph algorithm collapses the APIGW and Lambda errors down to `node-rds-01`.

2. Runbook Vector Search (RAG)

Gargi feeds the `cloudWatchLogs30m` error lines (e.g. `FATAL: remaining connection slots are reserved`) into her vector database to verify that her embedding search correctly retrieves the RDS Runbook and outputs `UPGRADE_RDS_INSTANCE`.

C. How YOU (Krish) Get Fresh or Live Data for Her:

Method 1: The CLI Tool

Run anytime from the terminal:
`npm run simulate -- --scenario container_crashloop --save`
Exports a freshly timestamped JSON file ready to send to Gargi.

Method 2: The Web Dashboard

Open `http://localhost:4000`. Click any of the 5 scenario buttons or click **Live AWS Trigger** to enter a real AWS resource ARN, then click **Download JSON** or **Copy JSON**.

Method 3: Live User AWS Query

Call the backend API with the user's AWS credentials/role:
`POST /api/aws/live-collect`
It automatically queries the real CloudWatch logs and CloudTrail from that AWS account and returns the formatted payload!